

Tarea UT3 Diseño y realización de pruebas

Abraham Quintana Micó

Tarea 1 utilizando NetBeans para depurar.

Ln 23 el texto a imprimir no está bien formado y el ide avisa de error de sintaxis:

Mal: `System.out.println("Agrupando " + numElementos elementos");`

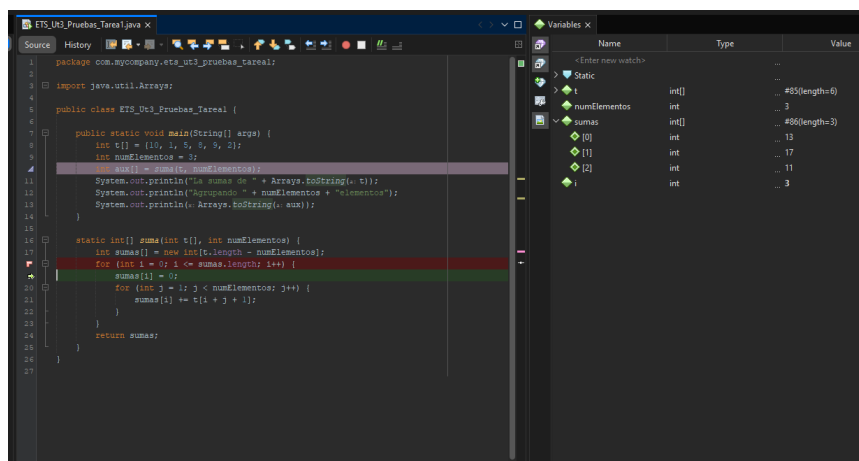
Corregido: `System.out.println("Agrupando " + numElementos + "elementos");`

Excepción lanzada al ejecutar el programa tras resolver el problema de sintaxis:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
    at com.mycompany.ets_ut3_pruebas_tareal.ETS_Ut3_Pruebas_Tareal.suma(ETS_Ut3_Pruebas_Tareal.java:19)
    at com.mycompany.ets_ut3_pruebas_tareal.ETS_Ut3_Pruebas_Tareal.main(ETS_Ut3_Pruebas_Tareal.java:10)
Command execution failed.
```

Avisa de que al ejecutar el método `suma()` intenta insertar un valor en un espacio del array cuya posición sobrepasa el tamaño del mismo.

Breakpoints en la línea anterior a la que lanza la excepción, y debug (añadiendo como condición que `i >= sumas.length`):



LN 17 Tamaño del array

Mal: `int sumas[] = new int[t.length - numElementos];`

Corregido: `int sumas[] = new int[t.length - (numElementos-1)];`

Ln 18 Recorrido condicion del for

Mal: `for (int i = 0; i <= sumas.length; i++) {`

Corregido: `for (int i = 0; i <= sumas.length - 1; i++) {`

Ln 20 Inicialización de la variable j

Mal: `for (int j = 1; j < numElementos; j++) {`

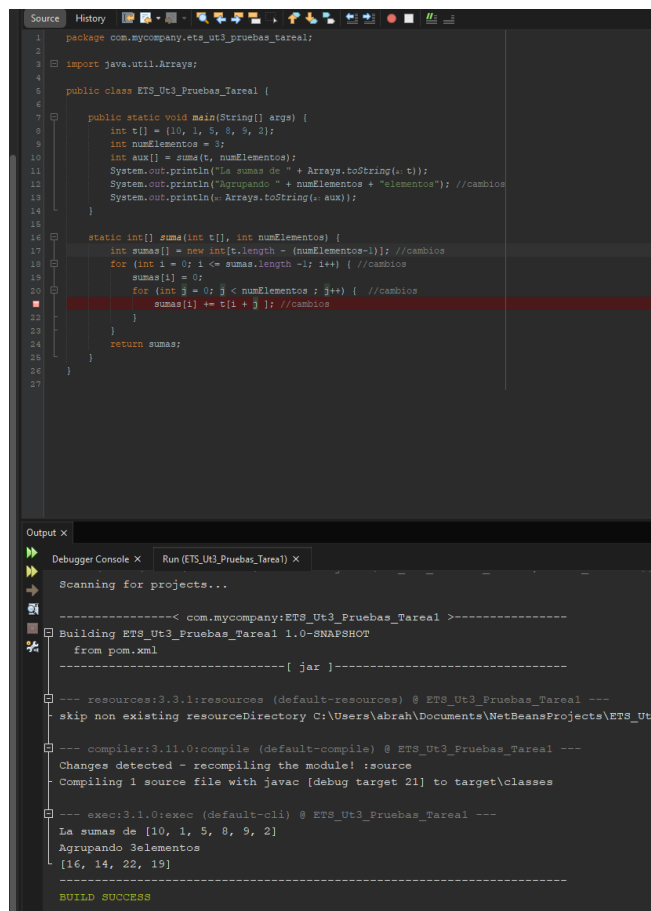
Corregido: `for (int j = 0; j < numElementos; j++) {`

Ln 21 Inserción en el array a devolver:

Mal: `sumas[i] += t[i+j+1];`

Corregido: `sumas[i] += t[i+j];`

Ejecución del código ya corregido y salida por consola:



The screenshot displays an IDE with a Java source file and its execution output. The source code defines a class `ETS_Ut3_Pruebas_Tareal` with a `main` method and a static `suma` method. The `main` method initializes an array `t` with values `[10, 1, 5, 8, 9, 2]`, calculates the number of elements, and prints the array and the number of elements. The `suma` method calculates the sum of the first `numElementos` elements of the array `t` and returns the result. The output window shows the execution of the code, including the compilation and execution steps, and the final output of the program.

```
1 package com.mycocompany.ets_ut3_pruebas_tareal;
2
3 import java.util.Arrays;
4
5 public class ETS_Ut3_Pruebas_Tareal {
6
7     public static void main(String[] args) {
8         int t[] = {10, 1, 5, 8, 9, 2};
9         int numElementos = 3;
10        int aux[] = suma(t, numElementos);
11        System.out.println("La sumas de " + Arrays.toString(t));
12        System.out.println("Agrupando " + numElementos + " elementos"); //cambios
13        System.out.println( Arrays.toString(aux));
14    }
15
16    static int[] suma(int t[], int numElementos) {
17        int sumas[] = new int[t.length - (numElementos-1)]; //cambios
18        for (int i = 0; i < sumas.length -1; i++) { //cambios
19            sumas[i] = 0;
20            for (int j = 0; j < numElementos - 1; j++) { //cambios
21                sumas[i] += t[i + j]; //cambios
22            }
23        }
24        return sumas;
25    }
26
27 }
```

Output x

Debugger Console x Run (ETS_Ut3_Pruebas_Tareal) x

Scanning for projects...

-----< com.mycocompany:ETS_Ut3_Pruebas_Tareal >-----

Building ETS_Ut3_Pruebas_Tareal 1.0-SNAPSHOT

from pom.xml

-----[jar]-----

--- resources:3.3.1:resources (default-resources) @ ETS_Ut3_Pruebas_Tareal ---

skip non existing resourceDirectory C:\Users\abrah\Documents\NetBeansProjects\ETS_Ut3

--- compiler:3.11.0:compile (default-compile) @ ETS_Ut3_Pruebas_Tareal ---

Changes detected - recompiling the module! :source

Compiling 1 source file with javac [debug target 21] to target\classes

--- exec:3.1.0:exec (default-cli) @ ETS_Ut3_Pruebas_Tareal ---

La sumas de [10, 1, 5, 8, 9, 2]

Agrupando 3 elementos

[16, 14, 22, 19]

BUILD SUCCESS

Conclusión:

El debug permite al programador seguir la secuencia de procesos e ir revisando como cambian las variables línea a línea, e incluso una de las mejores herramientas del debug, es la evaluación de expresiones, lo que permite comprobar exactamente donde está el problema (aunque solo sea una condición o una operación que se realice dentro de una línea) lo que permite incluso discriminar que proceso dentro de una misma línea es el que no está dando el valor esperado.

Tarea 2: Elabora unas series de pruebas unitarias en NetBeans

* Explicación del código:

Unifica dos arrays ordenados de enteros en uno resultante, ordenando sus valores de menor a mayor.

De manera que va por posiciones comprobando en ambos arrays que elemento va antes en el array resultante.

* Test creados según se especifica en el ejercicio:

```
package com.mycompany.ets_ut3_pruebas_tarea2;

import java.util.Arrays;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ETS_Ut3_Pruebas_Tarea2Test {

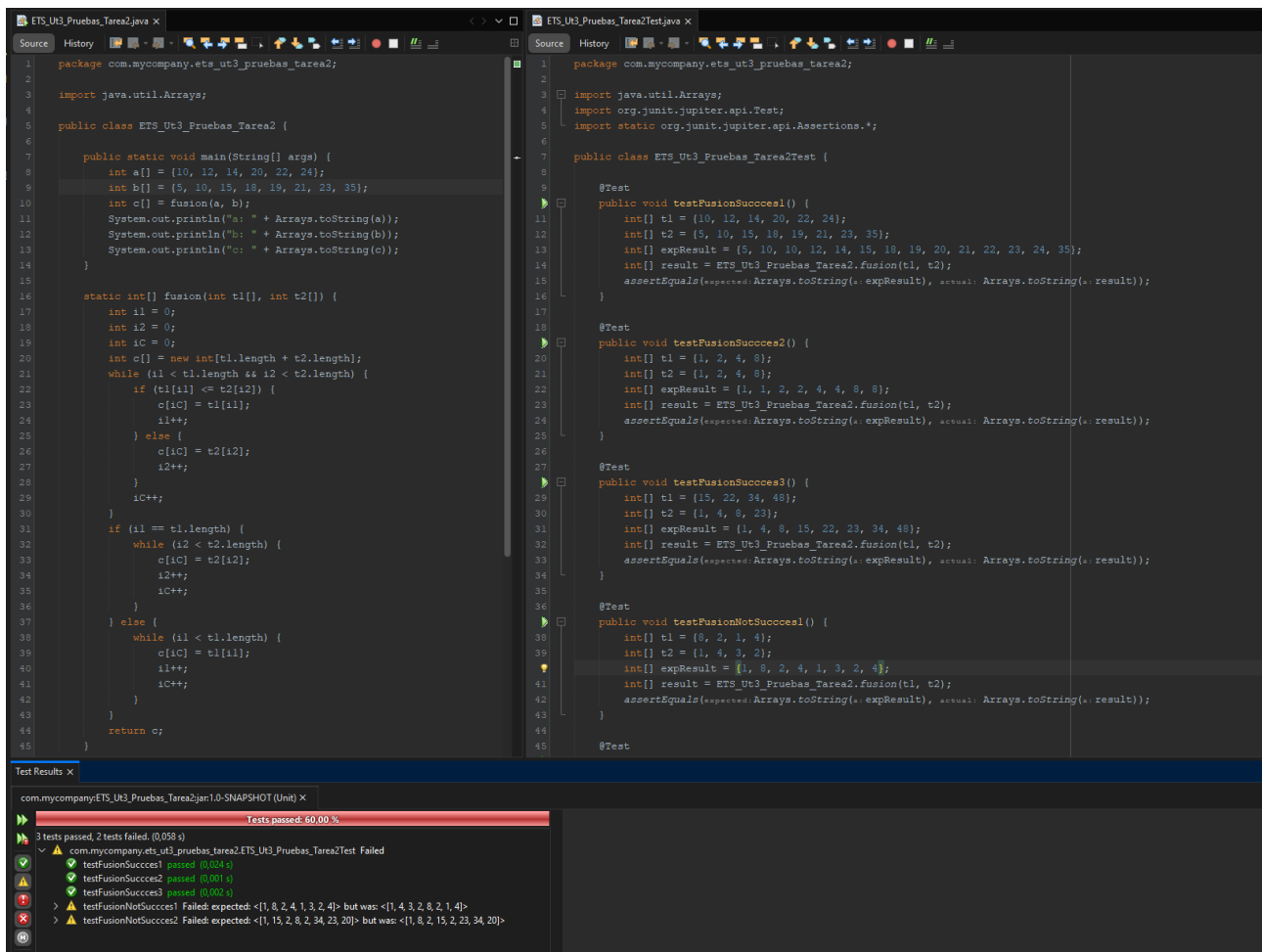
    @Test
    public void testFusionSuces1() {
        int[] t1 = {10, 12, 14, 20, 22, 24};
        int[] t2 = {5, 10, 15, 18, 19, 21, 23, 35};
        int[] expectedResult = {5, 10, 10, 12, 14, 15, 18, 19, 20, 21, 22, 23, 24, 35};
        int[] result = ETS_Ut3_Pruebas_Tarea2.fusion(t1, t2);
        assertEquals(Arrays.toString(expResult), Arrays.toString(result));
    }

    @Test
    public void testFusionSuces2() {
        int[] t1 = {1, 2, 4, 8};
        int[] t2 = {1, 2, 4, 8};
        int[] expectedResult = {1, 1, 2, 2, 4, 4, 8, 8};
        int[] result = ETS_Ut3_Pruebas_Tarea2.fusion(t1, t2);
        assertEquals(Arrays.toString(expResult), Arrays.toString(result));
    }

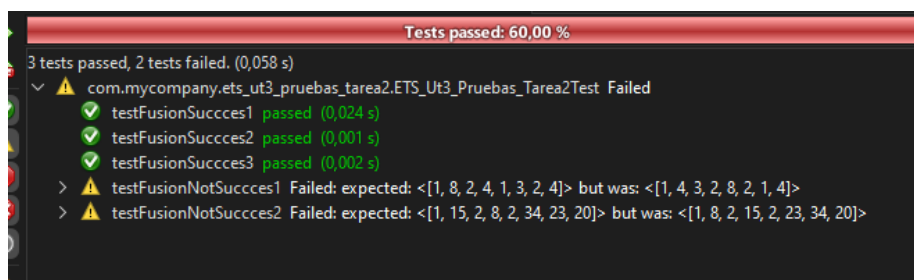
    @Test
    public void testFusionSuces3() {
        int[] t1 = {15, 22, 34, 48};
        int[] t2 = {1, 4, 8, 23};
        int[] expectedResult = {1, 4, 8, 15, 22, 23, 34, 48};
        int[] result = ETS_Ut3_Pruebas_Tarea2.fusion(t1, t2);
        assertEquals(Arrays.toString(expResult), Arrays.toString(result));
    }

    @Test
    public void testFusionNotSuces1() {
        int[] t1 = {8, 2, 1, 4};
        int[] t2 = {1, 4, 3, 2};
        int[] expectedResult = {1, 8, 2, 4, 1, 3, 2, 4};
        int[] result = ETS_Ut3_Pruebas_Tarea2.fusion(t1, t2);
        assertEquals(Arrays.toString(expResult), Arrays.toString(result));
    }

    @Test
    public void testFusionNotSuces2() {
        int[] t1 = {15, 2, 34, 20};
        int[] t2 = {1, 8, 2, 23};
        int[] expectedResult = {1, 15, 2, 8, 2, 34, 23, 20};
        int[] result = ETS_Ut3_Pruebas_Tarea2.fusion(t1, t2);
        assertEquals(Arrays.toString(expResult), Arrays.toString(result));
    }
}
```



* Resultados de los test:



* **Conclusión:**

Los test unitarios permiten al desarrollador comprobar método a método, que el código devuelve el resultado esperado o, que no lo devuelve cuando se está comprobando que da un error concreto por ejemplo como en los 2 test que deben dar error del ejercicio.

Lo mas correcto sería usar un `assertNotEquals` en lugar de el `assertEquals`. Ya que lo que se espera realmente es que no sean iguales y, Maven o gradle no será capaces de compilar el proyecto salvo que nos saltemos los test en el Lifecycle.